

QA Automation Workshop — Complete Cheatsheet

Who this is for: You're a QA beginner sitting down with Postman for the first time. Every step tells you what to do, how to do it, and why it matters.

Table of Contents

1. [Setup & Your First API Call](#)
 2. [Seed the Data & Explore CRUD](#)
 3. [POST & the Validation Wall](#)
 4. [PUT vs PATCH — Full Replace vs Partial Update](#)
 5. [Writing Your First Test Script](#)
 6. [Reset Between Runs — Setup & Teardown](#)
 7. [Collection Runner & Automated Suite](#)
 8. [Data-Driven Testing with CSV](#)
 9. [Bug Hunting on /buggy Endpoints](#)
 10. [Newman & CLI Runs](#)
 11. [Quick Reference Tables](#)
-

Key Terms You'll Use All Day

Term	What It Means
API	Application Programming Interface — a set of URLs (endpoints) a server exposes so programs can send data to it and get data back, without needing a browser or UI.
Endpoint	A specific URL path that does one job, e.g. /keys or /healthcheck.
HTTP Method	The verb that tells the server what you want to do: GET (read), POST (create), PUT (full replace), PATCH (partial update), DELETE (remove).
Request	What you send to the server: a method, a URL, optional headers, and an optional body.
Response	What the server sends back: a status code, headers, and a body (usually JSON).
Status Code	A 3-digit number telling you what happened. 200 = OK, 201 = Created, 400 = Bad Request, 404 = Not Found, 409 = Conflict, 422 = Validation Error, 500 = Server Error.
JSON	JavaScript Object Notation — the data format used in request and response bodies. Objects use {}, arrays use [].
Header	Metadata sent alongside a request or response. Content-Type: application/json tells the server "my body is JSON."
Collection	A saved group of requests in Postman — like a folder of test cases.
Environment	A set of variables (like baseUrl) in Postman that you can swap without editing every request.
Assertion	A check in your test script: "I expect this status code to be 200." If the check fails, the test fails.
CRUD	Create, Read, Update, Delete — the four basic operations on any data resource.
Error Envelope	A consistent JSON shape the API always uses for errors, so you always know where to find the error message.

Step 1 — Setup & Your First API Call

What You'll Learn

How to create a Postman collection from scratch, set up an environment with variables, add your very first request, and send it.

Why This Matters

In real-world QA, you build your own test collections from documentation and exploration. Nobody hands you a ready-made suite. You also never hardcode URLs — environments let you point the same tests at localhost, staging, or production by changing one variable. Learning to set this up yourself is the foundation of every Postman project you'll ever work on.

How To Do It

1.1 — Make sure the API is running

Your instructor has started the server. It's running at <http://localhost:8000>. You can verify by opening that URL in a browser — you should see the Swagger docs page at <http://localhost:8000/docs>.

What is Swagger? Swagger (also called OpenAPI docs) is an auto-generated interactive page that lists every endpoint the API exposes, what methods it accepts, what request bodies it expects, and what responses it returns. It's your map of the API. Open it now and keep it in a browser tab — you'll reference it throughout the workshop.

1.2 — Open Postman and create a new Collection

A collection is a container that holds all your requests, organized into folders. Think of it as a project folder for all the tests related to one API.

1. Open Postman (desktop app or web version).
2. In the left sidebar, click the + (plus) button next to "Collections", or click New → Collection.
3. Name it: Test-API Workshop.
4. Click Create (or just press Enter).

You now have an empty collection in the sidebar. Every request you build today goes inside it.

Why a collection and not loose requests? Loose requests in Postman are like unsaved files — they disappear when you close the tab. A collection saves everything, lets you organize with folders, run requests in sequence, and export the whole thing for CI/CD later.

1.3 — Create an Environment

An environment is a set of key-value variables. You'll create one variable: `baseUrl`.

1. In the top-right corner of Postman, click the Environment dropdown (it probably says "No Environment").
2. Click the eye icon (👁) next to the dropdown, then click Add (or go to Environments in the left sidebar → +).
3. Name the environment: Test-API Local.
4. In the variable table, add one row:
 - Variable: `baseUrl`
 - Type: default
 - Initial Value: `http://localhost:8000`
 - Current Value: `http://localhost:8000`
5. Click Save (Ctrl+S / Cmd+S).
6. Back in the top-right dropdown, select Test-API Local as your active environment.

Initial Value vs Current Value: Initial Value is what gets shared when you export the environment (safe for teams). Current Value is your local override — it's what Postman actually uses at runtime and is never shared. For this workshop they'll be the same. In a real team, you might have your personal API key in Current Value and a placeholder in Initial Value.

What is `{{baseUrl}}`? Anywhere you type `{{baseUrl}}` in a request URL, Postman replaces it at runtime with the value from your active environment. If you later test against a deployed server at `https://api.staging.example.com`, you change the variable once — not every request. This is why environments exist.

1.4 — Create your first request: GET /healthcheck

1. In the sidebar, hover over your Test-API Workshop collection → click the three dots (⋮) → Add request.
2. Name it: GET /healthcheck.
3. In the request tab that opens:
 - The method dropdown (left of the URL bar) should say GET. If not, click it and select GET.
 - In the URL bar, type: `{{baseUrl}}/healthcheck`
4. Click Send.

5. Look at the bottom panel — the Response section:

- Status: 200 OK (shown in green).
- Body: a JSON response confirming the server is alive.
- Time: how long the request took in milliseconds.
- Size: the response size in bytes.

Reading the Postman response panel: The bottom section has several tabs. Body shows the response content (switch between Pretty/Raw/Preview). Headers shows response headers. Test Results shows pass/fail for any scripts (empty for now). The status code, time, and size are displayed at the top-right of the response area.

6. Press Ctrl+S (Cmd+S on Mac) to save the request.

✅ Checkpoint: You just created a collection, an environment, your first request, and made your first API call. You sent a GET request, the server responded with 200, and you read the response. Everything else today builds on exactly this workflow: create a request, configure it, send it, read the response.

1.5 — Explore two more GET endpoints

Practice the workflow by adding two more requests to your collection:

1. Add request → Name it GET /version → Method: GET → URL: {{baseUrl}}/version → Send → Save.
2. Add request → Name it GET /stats → Method: GET → URL: {{baseUrl}}/stats → Send → Save.

Look at each response. /version tells you the API version. /stats shows how many items are in each data store (probably all zeros right now). You now have three working requests in your collection.

Key Concepts from This Step

- Collection = a saved group of related requests. You created one from scratch.
 - Environment = a set of variables (like baseUrl). You created one and activated it.
 - {{baseUrl}} = a variable placeholder that Postman replaces at runtime. You typed it in a URL.
 - GET is a read-only method. It asks the server for data and never changes anything.
 - 200 OK means "your request was understood and completed successfully."
 - Ctrl+S saves your request into the collection. Always save after making changes.
-

Step 2 — Seed the Data & Explore CRUD

What You'll Learn

How to populate the API with sample data, read it back using GET requests, delete an item, and observe what happens when you try to read something that no longer exists.

Why This Matters

Before you can test an API, it needs data. The `/admin/seed` endpoint loads predictable, repeatable sample data. This is critical for testing — if every run starts with the same data, your test results are reproducible. A test that passes sometimes and fails sometimes because of random data is worthless.

How To Do It

2.1 — Create and send: POST `/admin/seed`

This is your first POST request. Unlike GET, POST sends data to the server (or in this case, triggers an action).

1. Hover over your collection → ... → Add request.
2. Name it: POST `/admin/seed`.
3. Change the method dropdown from GET to POST.
4. URL: `{{baseUrl}}/admin/seed`
5. You don't need a request body for this endpoint — it's a trigger-style POST.
6. Click Send.
7. You should get a 200 OK with a confirmation message.
8. Save the request (Ctrl+S).

Why POST, not GET? Seeding changes data on the server (it writes sample records into storage). Any operation that changes server state should use POST, PUT, PATCH, or DELETE — never GET. GET is strictly for reading. This distinction matters: browsers, caches, and proxies all treat GET as safe to repeat or cache. POST tells everyone "this changes something."

2.2 — Create and send: GET `/keys` (list all)

1. Add request → Name: GET `/keys` → Method: GET → URL: `{{baseUrl}}/keys`
2. Click Send.
3. You get back 200 OK with a JSON array of key objects.
4. Save the request.

Look at the response body carefully:

```
[  
  { "Id": 1, "Data": "ABCDEF", "DataType": "A" },  
  { "Id": 2, "Data": "123456", "DataType": "N" }  
]
```

Notice: the response is always an array, even if there's only one item or zero items. This is a design contract of this API. Every request body you send and every response body you receive will be a JSON array. Knowing the contract tells you what shape to assert on in your tests.

2.3 — Create and send: GET /keys/1 (single item)

1. Add request → Name: GET /keys/1 → Method: GET → URL: {{baseUrl}}/keys/1
2. Click Send.
3. You get back 200 OK with an array containing just the one item with Id 1.
4. Save the request.

Path parameter: The 1 in /keys/1 is a path parameter — it tells the server which specific resource you want. In Postman you can also use the :id syntax in the URL (e.g. {{baseUrl}}/keys/:id) and then fill the value in the Params tab below the URL bar. Both approaches work. Using :id + Params tab is cleaner when you want to change the ID often without editing the URL string.

2.4 — Create and send: DELETE /keys/1

1. Add request → Name: DELETE /keys/1 → Method: DELETE → URL: {{baseUrl}}/keys/1
2. Click Send.
3. You get 200 OK — the item is gone.
4. Save the request.

Now go back to your GET /keys/1 tab and click Send again:

Status: 404 Not Found

```
{  
  "status": 404,  
  "error": "Not Found",  
  "message": "Id 1 not found in keys store",
```

```
"detail": null
}
```

This is your first encounter with the error envelope. Every error from this API has the same four fields: status, error, message, detail. This consistency means you can write one reusable test to validate any error response.

2.5 — Delete the same key again

Go back to DELETE /keys/1 and click Send one more time.

You get 404 Not Found again — you can't delete something that doesn't exist. This is the expected behavior: DELETE is idempotent in error for this API (deleting a missing resource returns 404, not a silent success).

Your collection so far: At this point you should have 6 saved requests in your sidebar: GET /healthcheck, GET /version, GET /stats, POST /admin/seed, GET /keys, GET /keys/1, DELETE /keys/1. Each one was created by you from scratch. This is how real test suites are built — one request at a time, each verified before moving on.

Key Concepts from This Step

Concept	Explanation
Seeding	Loading known data before tests so results are predictable.
CRUD	Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE). You just did R and D.
Path Parameter	A variable segment in the URL like /keys/1 that identifies a specific resource.
404 Not Found	The server understood your request but the resource doesn't exist.
Error Envelope	The consistent JSON shape (status, error, message, detail) used for all errors.
Array Contract	This API wraps everything in arrays — requests and responses alike.

Step 3 — POST & the Validation Wall

What You'll Learn

How to create new resources with POST, and what happens when you send invalid data. This is the single most important step for understanding API testing — you'll trigger six different error scenarios and learn what each status code means.

Why This Matters

A QA engineer's job is not to confirm that things work — it's to find out how they break. Anyone can send a valid request. Your value is knowing all the ways a request can be invalid and verifying the API handles each one correctly.

How To Do It

3.1 — Create and send a valid POST

1. Add request → Name: POST /keys [valid] → Method: POST → URL: {{baseUrl}}/keys
2. Click the Body tab (below the URL bar).
3. Select the raw radio button.
4. In the dropdown to the right of raw (it says "Text" by default), change it to JSON. This sets the Content-Type: application/json header automatically.
5. Enter this body:

```
[{"Id": 100, "Data": "HELLO", "DataType": "A"}]
```

6. Click Send.
7. You get 201 Created and the item is returned in an array.
8. Save the request.

Why the Body tab matters for POST: GET requests don't have a body — they only read data. POST, PUT, and PATCH send data in the body. The Body tab is where you compose that data. Choosing "raw" + "JSON" means you're writing the JSON yourself (as opposed to "form-data" or "x-www-form-urlencoded" which are for different content types). For this API, it's always raw JSON.

Why 201 and not 200? 201 Created specifically means "a new resource was created." It's more precise than 200 OK. Good APIs use specific status codes — and good testers assert on the specific code, not just "is it 2xx."

3.2 — Now break it: Empty array

Tip: Reusing a request for experiments. For the next several sub-steps, you don't need to create a new request each time. Just change the body in your POST /keys [valid] tab and click Send. You can also duplicate the request (right-click → Duplicate) if you want to save each variation separately — that's useful when you build your test suite later.

Send this body:

```
[]
```

Result: 400 Bad Request — the API requires at least one item.

400 Bad Request means "I understood your JSON, but it violates a business rule." The request is syntactically correct JSON but semantically wrong.

3.3 — Missing required field

```
[{"Id": 101, "DataType": "A"}]
```

Result: 422 Unprocessable Entity — the Data field is required.

422 vs 400: In this API, 422 comes from Pydantic (the data validation library). It means "your JSON has the right shape but fails schema validation" — missing fields, wrong types, out-of-range values. 400 is for business rule violations the API checks itself (empty array, duplicate ids).

3.4 — Invalid DataType

```
[{"Id": 101, "Data": "HELLO", "DataType": "X"}]
```

Result: 422 — DataType must be "A" or "N" for keys and ranges (or "A" / "Name" for matchscore).

3.5 — Empty Data string

```
[{"Id": 101, "Data": "", "DataType": "A"}]
```

Result: 422 — Data must be between 1 and 200 characters.

3.6 — Extra field

```
[{"Id": 101, "Data": "HELLO", "DataType": "A", "Foo": "bar"}]
```

Result: 422 — the model uses extra="forbid", meaning no unexpected fields are allowed.

Why forbid extra fields? In real systems, accepting unknown fields silently can mask bugs. If a client sends "Dataa": "HELLO" (typo), a permissive API would silently ignore it and create a record with no Data. Strict validation catches these mistakes.

3.7 — Duplicate ID (already exists in store)

Send the original valid request again:

```
[{"Id": 100, "Data": "HELLO", "DataType": "A"}]
```

Result: 409 Conflict — Id 100 already exists from step 3.1.

409 Conflict means "your request conflicts with the current state of the server." The data is valid, but it can't be applied because it would create a duplicate.

3.8 — Duplicate IDs within the same body

```
[
```

```
  {"Id": 200, "Data": "AAA", "DataType": "A"},
```

```
{ "Id": 200, "Data": "BBB", "DataType": "N" }  
]
```

Result: 400 Bad Request — two items in the same request have the same Id.

3.9 — Duplicate content (different IDs, same Data+DataType)

```
[  
  { "Id": 201, "Data": "SAME", "DataType": "A" },  
  { "Id": 202, "Data": "SAME", "DataType": "A" }  
]
```

Result: 400 Bad Request — the business rule says you can't have two records with identical Data and DataType, even if their Ids are different.

Status Code Summary from This Step

Code	Meaning	You Triggered It By
201	Created	Sending a valid, unique item
400	Bad Request	Empty array, duplicate IDs in body, duplicate content
409	Conflict	ID already exists in the store
422	Unprocessable Entity	Missing field, wrong type, empty string, extra field

Step 4 — PUT vs PATCH

What You'll Learn

The difference between full replacement (PUT) and partial update (PATCH), and what happens when your path ID doesn't match your body ID.

Why This Matters

PUT and PATCH are both "update" operations, but they behave very differently. Confusing them is a common source of real bugs. As a QA, you need to test both and know what each expects.

How To Do It

4.1 — PUT (full replacement)

PUT replaces the entire resource. You must send every field.

1. Add request → Name: PUT /keys/100 → Method: PUT → URL: {{baseUrl}}/keys/100

2. Body → raw → JSON. Set the body to:

```
[{"Id": 100, "Data": "WORLD", "DataType": "N"}]
```

3. Click Send.
4. 200 OK — the item is fully replaced. Data changed from "HELLO" to "WORLD", DataType changed from "A" to "N".

4.2 — PUT with mismatched ID

Now change the body Id to something different from the path:

```
[{"Id": 999, "Data": "WORLD", "DataType": "N"}]
```

URL is still /keys/100, but body says Id 999.

Result: 400 Bad Request — path-body ID mismatch.

Why this check? Without it, a client could accidentally overwrite the wrong resource. The API forces you to be explicit: the path says which resource, the body confirms it.

4.3 — PATCH (partial update)

PATCH only changes the fields you include. Everything else stays the same.

1. Add request → Name: PATCH /keys/100 → Method: PATCH → URL: {{baseUrl}}/keys/100
2. Body → raw → JSON. Set the body to:

```
{"Data": "PATCHED"}
```

Note: PATCH body is a plain object, not an array.

3. Click Send.
4. 200 OK — only Data changed. DataType is still "N" from the PUT.

4.4 — PATCH leaving fields untouched

Send a PATCH with just:

```
{"DataType": "A"}
```

Now DataType is "A" again but Data is still "PATCHED". Only what you send gets changed.

PUT vs PATCH Quick Reference

Aspect	PUT	PATCH
Body shape	Full resource in an array [{...}]	Partial object {...} (no array)
Required fields	All fields mandatory	Only the fields you want to change
ID rule	Body Id must match path Id	No Id required in body
Effect	Complete replacement	Merge changes into existing
If you omit a field	Error or field set to default	Field keeps its current value

Step 5 — Writing Your First Test Script

What You'll Learn

How to add automated assertions to your Postman requests so that "pass" or "fail" is determined by code, not by your eyes.

Why This Matters

Manual testing means a human looks at a response and decides if it's correct. Automated testing means code checks it. Manual testing doesn't scale — you can't eyeball 200 responses after every code change. Automated assertions run in seconds and catch regressions the moment they appear.

Theory: What Is a Test Script in Postman?

In Postman, every request has a Scripts tab (previously called "Tests" in older versions). Inside it, there's a Post-response section. Code you write here runs immediately after the response comes back. You use the pm (Postman) object to write assertions.

The basic pattern:

```
pm.test("Description of what you're checking", function () {  
    // one or more assertions here
```

```
});
```

If any assertion inside the function fails, that test is marked red. If all pass, it's marked green.

How To Do It

5.1 — Test the healthcheck

1. Open your GET /healthcheck request (click it in the sidebar).
2. Click the Scripts tab (next to Body, Headers, etc.) → select Post-response.
3. Type:

```
pm.test("Status code is 200", function () {  
  pm.response.to.have.status(200);  
});
```

4. Click Send.
5. Look at the Test Results tab (bottom panel, next to Body). You should see a green  "Status code is 200."

5.2 — Test a successful POST

Reset data first: open your POST /admin/reset request (create one if you haven't: POST → {{baseUrl}}/admin/reset) and click Send. Then send POST /admin/seed to reload sample data. Now open your POST /keys [valid] request with body [{"Id": 100, "Data": "HELLO", "DataType": "A"}].

Add these scripts:

```
// Check the status code
```

```
pm.test("Status code is 201", function () {  
  pm.response.to.have.status(201);  
});
```

```
// Check the response is an array
```

```
pm.test("Response body is an array", function () {  
  const body = pm.response.json();  
  pm.expect(body).to.be.an("array");  
});
```

```
// Check the array is not empty

pm.test("Response array is not empty", function () {

  const body = pm.response.json();

  pm.expect(body.length).to.be.greaterThan(0);

});
```

```
// Check the returned Id matches what we sent

pm.test("Returned Id matches sent Id", function () {

  const body = pm.response.json();

  pm.expect(body[0].Id).to.eql(100);

});
```

```
// Check all expected fields are present

pm.test("Response has all required fields", function () {

  const item = pm.response.json()[0];

  pm.expect(item).to.have.property("Id");

  pm.expect(item).to.have.property("Data");

  pm.expect(item).to.have.property("DataType");

});
```

Send it. You should see five green checkmarks.

5.3 — Test a negative case

Create a new request: Add request → Name: POST /keys [empty array] → Method: POST → URL: {{baseUrl}}/keys → Body: raw JSON → []

Add:

```
pm.test("Status code is 400 for empty array", function () {

  pm.response.to.have.status(400);

});
```

```
pm.test("Error envelope has all fields", function () {  
  const body = pm.response.json();  
  pm.expect(body).to.have.property("status");  
  pm.expect(body).to.have.property("error");  
  pm.expect(body).to.have.property("message");  
  pm.expect(body).to.have.property("detail");  
});
```

```
pm.test("Error status matches HTTP status", function () {  
  const body = pm.response.json();  
  pm.expect(body.status).to.eql(400);  
});
```

Send it. Three green checkmarks. You've just written your first negative test — a test that confirms the API rejects bad input correctly.

Common Assertion Methods Reference

// Status code

```
pm.response.to.have.status(200);  
pm.response.to.have.status(201);
```

// Response time (performance check)

```
pm.test("Response under 500ms", function () {  
  pm.expect(pm.response.responseTime).to.be.below(500);  
});
```

// Body is JSON

```
pm.response.to.be.json;
```

```
// Value checks

pm.expect(value).toEqual(expected);    // deep equality
pm.expect(value).toEqual(expected);    // strict equality
pm.expect(value).toBe.a("string");     // type check
pm.expect(value).toBe.an("array");     // type check
pm.expect(value).toHaveProperty("key"); // property exists
pm.expect(value).toContain("substring"); // string/array contains
pm.expect(value).toBe.true;            // boolean check
pm.expect(value).toBeGreaterThan(0);   // numeric comparison
pm.expect(array).toHaveLength(3);     // array/string length
```

Step 6 — Reset Between Runs

What You'll Learn

How and why to reset API data between test runs, establishing the concept of test setup and teardown.

Why This Matters

Imagine you run your POST test and it creates Id 100. You run it again — 409 Conflict because Id 100 already exists. Your test didn't break; the data was dirty from the previous run. This is the single most common cause of "flaky tests." The solution: always start from a known state.

How To Do It

6.1 — Create and send: POST /admin/reset

If you haven't created this request yet:

1. Add request → Name: POST /admin/reset → Method: POST → URL: {{baseUrl}}/admin/reset
2. Click Send.

This clears all three stores (keys, ranges, matchscore) to empty. Now GET /keys returns [].

6.2 — Create and send: POST /admin/reset/keys

1. Add request → Name: POST /admin/reset/keys → Method: POST → URL: {{baseUrl}}/admin/reset/keys
2. Click Send.

This clears only the keys store. The ranges and matchscore stores are untouched.

6.3 — Organize with folders

Your collection has grown — it's time to organize. You can create folders inside a collection and drag requests into them.

1. Hover over your collection → ... → Add folder. Name it Happy Path Tests.
2. Create another folder: Negative Tests.
3. Drag and drop your existing requests into the appropriate folder. Grab a request from the sidebar and drop it onto the folder name.

Organize like this:

Happy Path Tests

1. POST /admin/seed ← Setup: load known data
2. POST /keys [valid] ← Test
3. GET /keys ← Test
4. GET /keys/100 ← Test
5. PUT /keys/100 ← Test
6. PATCH /keys/100 ← Test
7. DELETE /keys/100 ← Test
8. POST /admin/reset ← Teardown: clean up

Setup and Teardown are standard testing concepts:

- Setup runs before your tests to create preconditions (seed data, create test users, etc.).
- Teardown runs after your tests to clean up (delete created data, reset state).
- Together they ensure every test run starts and ends the same way.

Step 7 — Collection Runner & Automated Suite

What You'll Learn

How to run an entire folder of requests in sequence with one click, and interpret the results.

Why This Matters

Until now you've been clicking Send on individual requests. That's manual testing with some assertions. The Collection Runner turns your collection into an automated test suite — it runs every request in order and gives you a pass/fail report. This is the foundation of test automation.

How To Do It

7.1 — Fill out your folders

In Step 6 you created Happy Path Tests and Negative Tests folders. Now make sure they're fully populated. Create any missing requests and drag them into the right folder:

Happy Path Tests

POST /admin/seed

POST /keys [valid body]

GET /keys

GET /keys/100

PUT /keys/100

PATCH /keys/100

DELETE /keys/100

POST /admin/reset

Negative Tests

POST /admin/seed

POST /keys [empty array] → expect 400

POST /keys [missing field] → expect 422

POST /keys [bad DataType] → expect 422

POST /keys [extra field] → expect 422

POST /keys [duplicate Id] → expect 409

GET /keys/99999 → expect 404

DELETE /keys/99999 → expect 404

POST /admin/reset

For each negative test request, create it the same way you've been doing: Add request → set method, URL, and body → Save. Make sure every request has test scripts (from Step 5).

7.2 — Open the Collection Runner

1. Click the Run button (▶) at the top of the collection or right-click the folder → Run folder.
2. You'll see a list of all requests in order.
3. Make sure the correct environment is selected in the top-right.
4. Click Run [Collection Name].

7.3 — Read the results

The runner shows every request and every test:

-  Green = passed
-  Red = failed

If everything is green, your automated suite passes. If something is red, click on it to see which assertion failed and what the actual response was.

7.4 — Intentionally break a test

Go to your POST /keys [valid body] request, change the expected status from 201 to 200:

```
pm.test("Status code is 200", function () { // Wrong on purpose!
    pm.response.to.have.status(200);
});
```

Run the suite again. The test fails red: "expected response to have status code 200 but got 201." Fix it back. This exercise proves that your assertions actually catch problems — they're not just decoration.

Key Concept: Test Execution Order

The Collection Runner runs requests top to bottom within each folder. Order matters! If your DELETE runs before your GET, the GET will 404 because the resource is already gone. Think of the sequence as a story: create → read → update → read again → delete → confirm deletion.

Step 8 — Data-Driven Testing with CSV

What You'll Learn

How to run the same request template dozens of times with different inputs from a CSV file, without duplicating requests.

Why This Matters

Imagine you need to test POST /matchscore with 20 different combinations of Data1, Data2, and DataType. You could create 20 separate requests — but that's tedious and unmaintainable. Data-driven testing means: one request template + one data file = as many test iterations as you have rows.

Theory: How It Works

Postman's Collection Runner accepts a data file (CSV or JSON). Each row in the file becomes one iteration. Column headers become variables you can reference with {{columnName}} in URLs, headers, and bodies.

Example CSV:

Id,Data1,Data2,DataType,expectedStatus

1,JOHN,JOHN,Name,201

2,JOHN,JANE,Name,201

3,ABC,ABC,A,201

4,,JOHN,Name,422

In your request body:

```
[{"Id": "{{Id}}", "Data1": "{{Data1}}", "Data2": "{{Data2}}", "DataType": "{{DataType}}"]
```

In your test script:

```
pm.test("Status matches expected: " + pm.iterationData.get("expectedStatus"), function () {  
    const expected = parseInt(pm.iterationData.get("expectedStatus"));  
    pm.response.to.have.status(expected);  
});
```

For each row in the CSV, Postman substitutes the values, sends the request, and runs the assertions.

How To Do It

8.1 — Get the CSV files

Your instructor will share two CSV files with you: data-driven-status-codes.csv and data-driven-matchscore.csv. Save them somewhere you can find them (e.g. your Desktop or project folder).

Open each one in a text editor (Notepad, VS Code, etc.) and look at the column headers and rows. Each row represents one test case. The column headers are the variable names you'll use in Postman.

8.2 — Create the data-driven request for status codes

You need a request that uses `{{variable}}` placeholders from the CSV columns.

1. Add request → Name: GET /error (data-driven) → Method: GET
2. URL: `{{baseUrl}}/error/{{statusCode}}`

Here `{{statusCode}}` will be replaced by the value from each CSV row.

3. Add a test script:

```
pm.test("Status matches expected: " + pm.iterationData.get("statusCode"), function () {  
  const expected = parseInt(pm.iterationData.get("statusCode"));  
  pm.response.to.have.status(expected);  
});
```

4. Save the request.

8.3 — Run with the CSV file

1. Open the Collection Runner (click Run ► on the collection or folder).
2. Select just this one request (uncheck others, or put it in its own folder).
3. In the runner, click Select File and choose data-driven-status-codes.csv.
4. You'll see the number of iterations matches the number of data rows.
5. Click Run.

Each iteration uses a different row. Watch the results — every row should show the correct status code.

8.4 — Create the data-driven request for matchscore

1. Add request → Name: POST /matchscore (data-driven) → Method: POST → URL:
`{{baseUrl}}/matchscore`
2. Body → raw → JSON:

```
[{"id": "{{id}}", "Data1": "{{Data1}}", "Data2": "{{Data2}}", "DataType": "{{DataType}}"}]
```

3. Add a test script, then Save.
4. Run it with the Collection Runner using data-driven-matchscore.csv.

8.4 — Accessing iteration data in scripts

// Read a value from the current CSV row

```
const expectedStatus = pm.iterationData.get("expectedStatus");
```

```
// Use it in assertions
```

```
pm.test("Status is " + expectedStatus, function () {  
    pm.response.to.have.status(parseInt(expectedStatus));  
});
```

```
// You can also check which iteration you're on
```

```
console.log("Iteration: " + pm.info.iteration);  
console.log("Iteration count: " + pm.info.iterationCount);
```

Key Concept: Why Data-Driven?

Approach	Requests to Maintain Test Cases	
One request per case	20	20
Data-driven (1 template + CSV)	1	As many rows as you want

When a field name changes, you update one request template — not 20.

Step 9 — Bug Hunting on /buggy Endpoints

What You'll Learn

How to compare buggy endpoints against known-good endpoints, identify defects, and write tests that prove the bug exists.

Why This Matters

This is the capstone exercise. In real QA work, you're given a feature, you learn how it should behave, and then you find where it doesn't. The `/buggy/*` endpoints are intentionally broken in specific ways. Your job is to figure out what's wrong — and prove it with an automated test.

How To Do It

9.1 — The approach: Compare buggy vs correct

You already know how the correct endpoints (`/keys`, `/ranges`, `/matchscore`) behave from Steps 2–4. The `/buggy/` versions should behave identically — but they don't. Your method:

1. Send the same request to the correct endpoint and note the response.
2. Send the same request to the buggy endpoint and note the response.
3. Spot the difference.
4. Write a test that passes on the correct endpoint but fails on the buggy one.

9.2 — The endpoints to investigate

Test each of these. Below are hints about what to look for, but try to find the bugs yourself first.

POST /buggy/keys

Try sending an empty array [].

- What does POST /keys return for []? → 400.
- What does POST /buggy/keys return for []? → ???

Write a test:

```
pm.test("Empty array should be rejected with 400", function () {  
  pm.response.to.have.status(400);  
});
```

This test passes on /keys and fails on /buggy/keys — that's your proof of the defect.

POST /buggy/matchscore

Send a valid matchscore request and compare the response to POST /matchscore.

Look carefully at the Data1 and Data2 fields in the response. Are they where you expect them?

```
pm.test("Data1 in response matches Data1 in request", function () {  
  const sent = JSON.parse(pm.request.body.raw);  
  const received = pm.response.json()[0];  
  pm.expect(received.Data1).to.eql(sent[0].Data1);  
});
```

GET /buggy/matchscore/{id}

Try requesting a matchscore ID that doesn't exist (e.g., /buggy/matchscore/999999).

- What does GET /matchscore/99999 return? → 404.
- What does GET /buggy/matchscore/99999 return? → ???

```
pm.test("Missing Id should return 404", function () {  
  pm.response.to.have.status(404);  
});
```

DELETE /buggy/ranges/{id}

Delete a range ID that doesn't exist.

- What does DELETE /ranges/99999 return? → 404.
- What does DELETE /buggy/ranges/99999 return? → ???

```
pm.test("Deleting non-existent Id should return 404", function () {  
  pm.response.to.have.status(404);  
});
```

GET /buggy/echo

Just send a GET to this endpoint and examine the response.

Ask yourself: Should an API ever reveal its internal file paths or secret tokens to a caller?

```
pm.test("Response should NOT contain internal paths", function () {  
  const body = JSON.stringify(pm.response.json());  
  pm.expect(body).to.not.include("storage");  
});
```

```
pm.test("Response should NOT contain secret tokens", function () {  
  const body = JSON.stringify(pm.response.json());  
  pm.expect(body).to.not.include("token");  
});
```

9.3 — Document your findings

For each bug, record:

Field	Example
Endpoint	POST /buggy/keys

Steps to reproduce Send POST with body []

Field	Example
Expected result	400 Bad Request (matches /keys behavior)
Actual result	200 OK (empty array accepted)
Severity	Medium — allows creation of invalid state
Test script	pm.response.to.have.status(400); fails

This is a real bug report format. The ability to write a clear reproduction with expected vs actual behavior is one of the most important QA skills.

Step 10 — Newman & CLI Runs

What You'll Learn

How to run your entire Postman collection from the command line using Newman, without opening the Postman GUI.

Why This Matters

The Postman GUI is great for building and debugging tests. But in a real software team, tests need to run automatically — on every code change, in a CI/CD pipeline (Jenkins, GitHub Actions, GitLab CI, etc.), without a human clicking buttons. Newman is the tool that makes this possible: it takes the same collection and environment files and runs them in a terminal.

Theory: What Is Newman?

Newman is the official command-line runner for Postman collections. It reads the same .json collection and environment files you exported from Postman and executes every request, running all test scripts and reporting pass/fail results in the terminal.

How To Do It

10.1 — Install Newman (if not already installed)

```
npm install -g newman
```

npm is the Node.js package manager. The -g flag installs Newman globally so you can use it from any folder.

10.2 — Export your collection and environment from Postman

Everything you built today in Postman can be exported as JSON files that Newman understands.

1. In Postman, right-click your Test-API Workshop collection → Export → choose the latest format → Save as Test-API-Workshop.postman_collection.json.

2. Click the environment dropdown → gear icon (⚙) → select Test-API Local → Export → Save as Test-API-Local.postman_environment.json.
3. Put both files in a folder on your machine (e.g. ~/workshop/).

10.3 — Run the basic suite

Navigate to the folder where you saved the files, then run:

```
newman run Test-API-Workshop.postman_collection.json -e Test-API-Local.postman_environment.json
```

You'll see terminal output listing every request, every test, and a summary at the bottom:

	executed	failed
iterations	1	0
requests	15	0
test-scripts	15	0
assertions	42	0
total run duration: 1.2s		

10.4 — Run with a data file

```
newman run Test-API-Workshop.postman_collection.json -e Test-API-Local.postman_environment.json -d data-driven-matchscore.csv
```

Newman iterates over each CSV row, just like the Postman Collection Runner.

10.5 — Run with a specific folder

```
newman run Test-API-Workshop.postman_collection.json -e Test-API-Local.postman_environment.json --folder "Negative Tests"
```

10.6 — Generate an HTML report

```
npm install -g newman-reporter-html
```

```
newman run Test-API-Workshop.postman_collection.json -e Test-API-Local.postman_environment.json -r html
```

This creates an HTML file you can open in a browser — a shareable test report.

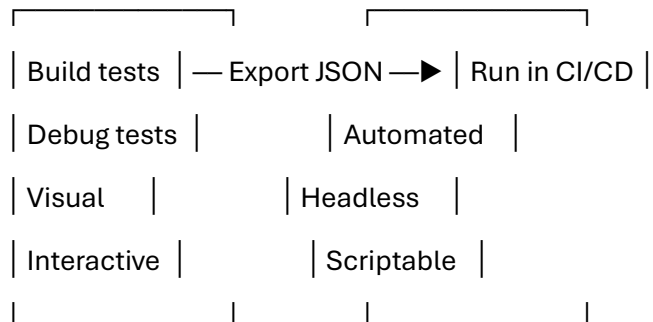
Newman Command Reference

Command / Flag	What It Does
<code>newman run <file></code>	Runs the collection
<code>-e <file></code>	Specifies the environment file
<code>-d <file></code>	Specifies a CSV or JSON data file for data-driven runs
<code>-n 5</code>	Runs the collection 5 times (iterations)
<code>--folder "Name"</code>	Runs only a specific folder
<code>-r html,cli</code>	Specifies reporters (cli is the default terminal output)
<code>--delay-request 500</code>	Adds a 500ms delay between requests
<code>--bail</code>	Stops on first failure
<code>--verbose</code>	Shows full request/response details

The Big Picture

Postman GUI

Newman CLI



You use Postman to create and debug. You use Newman to automate and integrate.

Quick Reference

HTTP Methods

Method	Purpose	Has Body?	Idempotent?
GET	Read resource(s)	No	Yes
POST	Create new resource	Yes	No
PUT	Full replacement	Yes	Yes
PATCH	Partial update	Yes	Yes
DELETE	Remove resource	No	Yes

Idempotent means: calling it once or ten times produces the same result. GET /keys/1 always returns the same item. DELETE /keys/1 the first time removes it; the second time it's already gone (404). POST is NOT idempotent — calling it twice creates two resources (or a 409 if Id clashes).

Status Codes Used in This API

Code	Name	When It Happens
200	OK	Successful GET, PUT, PATCH, DELETE
201	Created	Successful POST
400	Bad Request	Empty array, duplicate IDs in body, path-body ID mismatch, duplicate content
404	Not Found	Resource with that ID doesn't exist
409	Conflict	Trying to POST an ID that already exists
422	Unprocessable Entity	Pydantic validation failure (missing field, bad type, extra field, out-of-range value)
500	Internal Server Error	Server crashed (bug, unhandled exception)

Error Envelope Format

Every error response has this shape:

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Human-readable explanation of what went wrong",
  "detail": null
}
```

}

- status — mirrors the HTTP status code (always matches).
- error — the standard name for that status code.
- message — specific explanation of the problem.
- detail — additional context (often null, sometimes contains Pydantic validation details for 422s).

Validation Rules Cheat Sheet

Rule	What Happens If Violated
Body must be a non-empty JSON array (POST/PUT)	400
Id must be an integer	422
Id must be unique within the request body	400
Id must not already exist in the store (POST)	409
DataType must be "A" or "N" (keys/ranges)	422
DataType must be "A" or "Name" (matchscore)	422
Data, Data1, Data2 must be 1–200 characters	422
No extra fields allowed in the body	422
Same Data + DataType combination must not already exist	400
PUT: path Id must match body Id	400

Postman Scripting Cheat Sheet

// — Access response data —

const body = pm.response.json(); // parse JSON body

const status = pm.response.code; // numeric status code

const headers = pm.response.headers; // response headers

const time = pm.response.responseTime; // ms

// — Status assertions —

```
pm.response.to.have.status(200);
pm.response.to.have.status(201);

// —— Body structure ——

pm.expect(body).to.be.an("array");
pm.expect(body).to.have.lengthOf(1);
pm.expect(body[0]).to.have.property("Id");
pm.expect(body[0]).to.have.all.keys("Id", "Data", "DataType");

// —— Value assertions ——

pm.expect(body[0].Id).to.eql(100);
pm.expect(body[0].Data).to.eql("HELLO");
pm.expect(body[0].DataType).to.be.oneOf(["A", "N"]);

// —— String assertions ——

pm.expect(body.message).to.include("not found");
pm.expect(body.error).to.match(/Bad Request/);

// —— Negative assertions ——

pm.expect(body).to.not.be.empty;
pm.expect(body).to.not.have.property("secret");

// —— Error envelope check (reusable) ——

pm.test("Valid error envelope", function () {
  const err = pm.response.json();
  pm.expect(err).to.have.property("status");
  pm.expect(err).to.have.property("error");
```

```

    pm.expect(err).to.have.property("message");
    pm.expect(err).to.have.property("detail");
    pm.expect(err.status).to.eql(pm.response.code);
  });

  // —— Data-driven iteration data ——

  const val = pm.iterationData.get("columnName");
  const iter = pm.info.iteration;
  const total = pm.info.iterationCount;

  // —— Set/get variables ——

  pm.environment.set("createdId", body[0].Id); // save for later
  pm.environment.get("createdId");           // retrieve
  pm.collectionVariables.set("key", "value"); // collection scope

  // —— Performance ——

  pm.test("Response under 500ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
  });

  // —— Console logging (for debugging) ——

  console.log("Response body:", JSON.stringify(body, null, 2));

```

Endpoint Map

Method	Endpoint	Expected Status	Notes
GET	/healthcheck	200	Server alive check
GET	/version	200	API version

Method	Endpoint	Expected Status	Notes
GET	/stats	200	Store counts
POST	/admin/seed	200	Load sample data
POST	/admin/reset	200	Clear all stores
POST	/admin/reset/{store}	200	Clear one store (keys/ranges/matchscore)
GET	/error/{code}	{code}	Returns the code you ask for
GET	/error/boom	500	Unhandled exception
POST	/keys	201	Create key(s)
GET	/keys	200	List all keys
GET	/keys/{id}	200/404	Get one key
PUT	/keys/{id}	200/400/404	Replace a key
PATCH	/keys/{id}	200/404	Partial update
DELETE	/keys/{id}	200/404	Remove a key
POST	/ranges	201	Same CRUD pattern as keys
POST	/matchscore	201	Uses Data1, Data2 instead of Data
POST	/buggy/keys	???	 Find the defect
POST	/buggy/matchscore	???	 Find the defect
GET	/buggy/matchscore/{id}	???	 Find the defect
DELETE	/buggy/ranges/{id}	???	 Find the defect
GET	/buggy/echo	???	 Find the defect

Workshop Flow Summary

Step	Duration	What You Do
1. Setup & First Hit	15 min	Create collection, environment, {{baseUrl}}, first requests, hit /healthcheck
2. Seed & Explore CRUD	20 min	Create POST/GET/DELETE requests, seed data, observe 200 and 404
3. POST & Validation	30 min	Create items, break them 8 different ways, learn all status codes
4. PUT vs PATCH	15 min	Full replace vs partial update, path-body mismatch
5. First Test Scripts	30 min	Write assertions for positive and negative cases
6. Reset Between Runs	10 min	Create reset requests, organize folders, setup/teardown
7. Collection Runner	30 min	Automate the suite, interpret results
8. Data-Driven CSV	30 min	Create template requests, run against CSV data files
9. Bug Hunting	40 min	Find, prove, and document defects on /buggy endpoints
10. Newman CLI	20 min	Export collection, run everything headless from the terminal
Total	~4 hours	

Built for the 8th-semester QA Automation Workshop. Reset the API (POST /admin/reset) whenever things get messy. Break things on purpose — that's the whole point.